Scrimba Lesson Engine

Implementation Report

How it actually works

I split the flow into three jobs and deliberately keep them separate, so I can tune each on its own.

1. Write the script. One Claude call turns the user's question into a JSON array of 5–8 scenes. Each scene carries a `title`, the `narration` (what the voice says), a `visualBrief` (what an animator should draw), and a rough `durationSeconds`. I prompt Claude to open with a hook, build one idea per scene, and close with a takeaway, basically the shape of a good short explainer. (`server/pipeline/scriptGen.ts`)

2. Build each scene. For every scene I make a separate Claude call that returns a complete, standalone HTML document with all CSS and JS inline, no external anything. I run up to 3 at a time so the lesson comes together quickly. This prompt is the heart of the project, and it's where I spent most of my iteration (more on that below). (`server/pipeline/sceneGen.ts`)

3. Add the voice. I send each scene's narration to ElevenLabs and get back a base64 MP3. If that call fails, I just return an empty string, so the lesson still plays; it's just silent rather than broken. (`server/pipeline/tts.ts`)

I stream the pieces to the browser over Server-Sent Events. The server returns a `lessonId` instantly and kicks the pipeline off in the background; the player opens an SSE connection and receives `script_ready`, then a `scene_ready` for each finished scene, then `done`. Because the scenes generate in parallel they can arrive out of order, so I key everything by `index`.

Syncing the visuals to the audio

This part is fiddlier than it looks, because the scene animations live inside **sandboxed iframes**, so the player can't reach in and control them directly.

- **Audio is the clock.** When a scene starts, I play its narration through the Web Audio API and treat the audio's real length as the scene's true duration. When the audio ends, I advance. The script's `durationSeconds` is only a fallback for scenes that ended up silent. This keeps the visuals tied to the voice instead of drifting against a guessed timer. (`playScene` / `advance` in `client/src/player.ts`)
- **Pause/resume across the sandbox boundary.** Since I can't pause an iframe from outside, I inject a tiny controller at the very top of every scene that wraps `requestAnimationFrame`, `setTimeout`/`setInterval`, and the Web Animations API. The player posts `{__lessonCtrl: 'pause'|'resume'}` into the frame, and the controller freezes or thaws everything from the inside; timers remember their remaining time, animations pause in place. The audio pauses alongside it and resumes from the saved offset. (`SCENE_TIMING_CONTROL`)
- **Graceful stalling.** If the next scene hasn't generated yet when the current one ends, playback simply parks on the last frame and waits, then continues the moment that scene's `scene_ready` arrives. No blank flashes, no hard stops. (`waitingForNext` / `onSceneArrived`)
- **The progress track** mirrors all of this: each scene is a segment that shimmers while loading, fills left-to-right in sync with the audio while playing, and dims once played. Each segment is clickable to seek.

The iterative process: getting the visuals right

My first versions worked end-to-end but looked rough. When I let Claude draw freely, the scenes came out **cluttered**: SVGs piled on top of each other, labels overlapping the diagrams, elements drifting off the edges of the screen or sliding

behind the controls bar. Each scene was individually fine in isolation, but as a *lesson* it felt noisy and inconsistent; every scene had a different layout, different spacing, different sense of where to look.

So I stepped back and looked at it as an attention problem. I read up on how people actually watch explainer content like what holds attention and what makes them check out. The recurring lesson from how good **infographic / motion-explainer videos** are made is that they reuse a **small set of layouts** (a hero diagram, a split "visual + explanation," a big title card, a timeline, an A-vs-B comparison) and let the *content* change while the *frame* stays calm and predictable. The viewer's eye learns where to look once and then just follows the idea. The structure does the work of guiding attention.

That observation is what led me to use **templates.**

The templates

Rather than asking Claude "make a nice scene," I gave it a fixed menu of layout skeletons and told it to pick exactly one per scene, declare it on `<body data-template="...">`, and pour the content into that template's named zones. I define each template as a CSS Grid with named areas (in `server/pipeline/sceneGen.ts`):

Template	Shape	Best for
`diagram-focus`	label on top, one hero visual, caption below	a single concept or mechanism to study closely
`split-explain`	visual left, text right	a concept paired with its definition/properties
`title-reveal`	full-bleed centered statement	openings, closings, one big idea or number
`timeline`	header, a horizontal row of steps, caption	ordered sequences, stages, cause→effect
`comparison`	header, two equal columns, caption	A vs B, before/after, two contrasting ideas
`quad-grid`	header, then a 2×2 grid	recaps and "four parts of a whole" summaries

Why this helped so much:

1. **Consistency.** Every scene now shares the same grid logic and the same `:root` design tokens (one palette, one type scale, one spacing rhythm I paste verbatim into every scene). The lesson finally looked like *one* piece instead of eight unrelated pages.
2. **Attention.** Each template has a clear, single answer to "where do I look?" Text annotates the visual instead of fighting it, because the template keeps them in separate zones.
3. **Containment.** This is the big practical win. Every zone is contained. That means content physically *cannot* spill across a boundary; if something's too big it gets clipped inside its own zone instead of sprawling across the screen. It acts as gaurd for the prompts too, like instead of prompting "please don't overflow" it turned it into a structural guarantee.

I paired the templates with explicit **boundary rules** in the prompt (size rows in fr/%, never fixed px taller than the viewport; SVGs scale to their zone with `preserveAspectRatio`; keep clear of the bottom edge where the controls sit) and a **Scene Guard** that the player injects as a last line of defense: it hard-clips the document to the viewport and, if a scene's content is still too tall or wide, scales it down so nothing bleeds behind the controls. (`SCENE_GUARD` in `client/src/player.ts`)

Comparing the models: Opus 4.5 vs Opus 4.8

With the templates in place, the remaining variable was the model itself, and the difference here was bigger than I expected.

Opus 4.5 was going against the instructions. Even when I explicitly said *keep visuals simple, basic shapes, no intricate diagrams*, it would reach for complicated, detailed illustrations, busy compositions, the occasional axis chart, exactly the kind of clutter the templates were meant to prevent. It treated "simple" as a suggestion.

Opus 4.8 followed the same prompt much more faithfully. It stuck to clean, basic shapes, respected the template zones, and kept scenes calm and legible. The instructions I'd already written suddenly *worked*. The jump in output quality was large and immediate: the same prompt, the same templates, a noticeably better lesson.

My takeaway: the templates give a baseline the model can't easily break, and Opus 4.8 follows the intent of the prompt, not just the wording. Together they make the output consistently good instead of just working.

A late assumption: what “the output” actually is

For most of the build I worked under one assumption: the deliverable was the **experience inside the app**, where you type a question, the lesson streams in scene by scene, and you watch it play in the live player. I oriented everything around that. The “output” was the running app.

Late on, I reconsidered and landed on a different reading: the intended output is probably a **standalone, self-contained artifact**, a single HTML/DOM file that *is* the lesson and can be opened, shared, or hosted on its own, without the server, the API keys, or the streaming pipeline running. A lesson you can hand to someone, not just a session you can watch.

So I added an **export path** at the end: you can download a finished lesson as one self-contained `.html` file that bakes in every scene, its base64 narration audio, and a standalone port of the player engine (`server/export.ts`, `server/standalone/`). It plays offline when you just open the file. The embedded player reuses the same “audio is the clock” logic, so narration and visuals stay locked together exactly as they do live. The only concession is a click-to-start screen, because browsers won’t allow audio to begin without a user gesture. So, the live app is how I generate and preview a lesson, and the standalone file is how I deliver it. I produce the example lessons below through that export.

Example lessons

1. **What’s the difference between a hash map and a B-tree?** link
2. **How does the Norwegian parliament work?** link
3. **Why is the sky blue?:** link

What’s still rough, and where to push next

It’s not perfect. Sometimes an element still ends up too close to a boundary or crowds its neighbor, usually when a scene packs in too much. The fix isn’t more player-side patching; it’s the two things that already work:

- **Tighten the templates.** More specific zone sizing, clearer rules on how much content a zone should hold, and a few purpose-built templates for the cases that get squeezed today.
- **Improve the Scene Guards.** The auto-fit catches the worst overflows now; it could also detect crowding (not just overflow) and adjust spacing before scaling down.

In short: the structure is what makes this reliable. Better templates and stricter guards are the best way to keep the visuals clean.